Compression of a VGG-Style CNN on CIFAR-10 via Pruning and Quantization

Drake Weller
CSI-5140-12740 / CSI-4140-14165.202610
Oakland University
drakeweller@oakland.edu

April 13, 2026

Abstract

This work explores how much a VGG-style CNN trained on CIFAR-10 can be compressed without sacrificing classification performance. We apply two techniques from different compression families: global unstructured magnitude pruning and post-training static quantization. On the pruning side, we find that up to 90% of the model's weights can be zeroed out with barely any accuracy loss (90.56% vs. a 90.68% baseline). For quantization, converting the model from 32-bit floating point to 8-bit integers cuts its size by 4× and speeds up CPU inference by roughly 7.5×, again with no measurable drop in accuracy. An ablation study further reveals that how you distribute pruning across layers matters enormously—global pruning beats uniform layer-wise pruning by over 53 points at 95% sparsity—and that fine-tuning recovery after pruning follows qualitatively different patterns depending on how aggressively the model was pruned.

1 Introduction

Modern CNNs deliver impressive accuracy, but it tends to be too large for smaller devices such as phones, embedded systems, or other devices on the edge, where memory and compute are limited. Model compression helps by making networks smaller and faster while keeping them just as accurate.

We take a VGG-style CNN [1] trained on CIFAR-10 [**?**] and apply two different compression approaches:

1. **Unstructured magnitude pruning** [2], which zeros out the smallest weights in the network. if a weight is close to zero, activation is very little making them a non factor so removing will have little effect on the model accuracy.

2. **Post-training static quantization** [3], which represents weights and activations using 8-bit integers instead of 32-bit floats. This makes the

model close to 4× smaller and lets it use faster integer math at inference time.

Several findings stood out from our experiments. First, the model turns out to be heavily overparameterized, we can prune 90% of its weights and still maintain over 90% test accuracy. Second, global pruning (where we rank *all* weights and remove the smallest ones network-wide) works far better than pruning each layer by the same amount. Third, static quantization delivers real, measurable speedups on standard CPU hardware, unlike unstructured pruning which only reduces theoretical FLOPs. Finally, tracking the fine-tuning process after pruning reveals three distinct recovery regimes depending on how much was removed.

2 Compression Methods

Compression methods for neural networks generally fall into three families.

Pruning. The core idea goes back to early work on optimal brain damage [2], but gained modern traction with Han et al., who showed that magnitude based pruning can remove the majority of the weights from networks like AlexNet and VGG with little loss in accuracy. Other methods includes structured pruning [4], which removes entire filters rather than individual weights, making the resulting models faster on standard hardware, and the lottery ticket hypothesis [5], which mentions that dense networks contain sparse sub networks that could have been trained in isolation.

Quantization. Jacob et al. [3] showed how to train networks that run entirely in 8-bit integer arithmetic, achieving significant speedups on mobile hardware. Post-training approaches [6] skip the retraining step altogether, converting pre-trained FP32 models to INT8 using calibration data to set the quantization parameters.

Knowledge distillation. Hinton et al. [7] proposed training a compact "student" model to replicate the outputs of a larger "teacher." This is effective but requires designing a separate architecture, so we do not pursue it here.

Our work combines pruning and quantization, which compress models in fundamentally different ways: pruning removes parameters, while quantization shrinks the representation of each remaining parameter.

3 Methodology

3.1 Baseline Model

The starting point is a VGG-style CNN with eight convolutional layers organized into four blocks, followed by global average pooling and a small classifier head. Table 1 gives the full layout.

Global average pooling is used instead of having more blocks and more fully connected layers usual of VGG, this helps keeps the classifier lightweight, only

Table 1: Architecture of the VGG-style CNN. Every conv layer uses 3×3 filters with padding 1, followed by BatchNorm and ReLU. Dropout increases across blocks to regularize the deeper, higher-capacity layers.

Block	**Layers**	**Output Shape**	**Params**	**Dropout**
Block 1	Conv(3→64), Conv(64→64), MaxPool	$64 \times 16 \times 16$	38,720	0.1
Block 2	Conv(64→128), Conv(128→128), MaxPool	$128 \times 8 \times 8$	221,440	0.2
Block 3	Conv(128→256), Conv(256→256), MaxPool	$256 \times 4 \times 4$	885,248	0.3
Block 4	Conv(256→512), Conv(512→512), MaxPool	$512 \times 2 \times 2$	3,540,480	0.4
GAP	Adaptive Average Pool	512	0	—
FC1	Linear(512→256), ReLU	256	131,328	0.5
FC2	Linear(256→10)	10	2,570	—
Total			**∼4.82M**	

∼134K parameters in FC1 and FC2 combined. Progressive dropout, 0.1 in Block 1 up to 0.5 in the classifier, provides regularization that scales with layer capacity.

Training used Adam with an initial learning rate of 0.002, cosine annealing down to 10^{-4}, weight decay of 5×10^{-4}, and standard augmentation; random crops, horizontal flips, color jitter for 20 epochs. This produced a baseline test accuracy of **90.68%**.

3.2 Pruning

We use global unstructured L_1 pruning [2]. The procedure collects all weights from every Conv2d and Linear layer, ranks them by absolute value, and zeros out the bottom s-fraction, where s is the target sparsity. Because the ranking is global, larger layers with more redundancy naturally absorb a bigger share of the pruning, while smaller or more sensitive layers are relatively preserved.

After pruning, we fine-tune for 5 epochs with SGD , lr=10^{-4}, momentum 0.9, cosine schedule. The pruning masks stay fixed during fine-tuning and the zeroed weights are not allowed to grow back.

We test sparsity levels from 10% up to 95%.

3.3 Quantization

We use PyTorch's post-training static quantization pipeline to convert the model from FP32 to INT8. The process has three steps. First, we fuse consecutive Conv+BN+ReLU layers into single operations so that intermediate values don't need to be repeatedly quantized and dequantized. Second, we calibrate the quantization parameters by passing 50 batches of training data (∼6,400 images) through the model with observer modules that track activation ranges. Third, we convert the model to use INT8 arithmetic everywhere.

For comparison, we also test dynamic quantization, which is simpler (it

only quantizes Linear layers and computes activation scales on the fly) but less effective for conv-heavy architectures.

4 Experiments and Results

4.1 Pruning Results

Table 2 summarizes performance across sparsity levels. What stands out is how resilient the model is. At 80% sparsity, where four out of five weights have been removed, test accuracy actually increases slightly to 90.87%. The model doesn't start losing more accuracy until 90% sparsity, and even then the drop is small, 90.53% after fine-tuning. Only at 95% do we see a more notable decline, though fine-tuning still recovers the model to 89.92%.

Table 2: Pruning results. Before FT, is the accuracy right after zeroing out weights; After FT is the accuracy after 5 epochs of fine-tuning.

Sparsity	**Non-zero Params**	**Eff. FLOPs**	**Test (Before FT)**	**Test (After FT)**	**Train (After FT)**
0%	4,823,111	419.0M	90.68%	90.68%	94.38%
10%	4,341,405	403.0M	90.68%	90.82%	94.99%
30%	3,377,988	371.4M	90.68%	90.66%	94.92%
50%	2,414,570	336.6M	90.68%	90.85%	94.96%
70%	1,451,152	286.8M	90.68%	90.81%	94.89%
80%	969,444	246.1M	90.69%	90.87%	95.00%
90%	487,735	158.0M	90.38%	90.53%	94.87%
95%	246,880	93.5M	86.38%	89.92%	93.98%

One caveat: the FLOPs in Table 2 are effective FLOPs, counting only the non-zero multiply-accumulate operations. In practice, unstructured sparsity doesn't speed up inference on standard hardware because the tensor shapes remain unchanged—the GPU still processes every entry, including the zeros. Realizing actual speedup would require either structured pruning ,removing entire filters or specialized sparse computation engines.

Figure 1 plots this relationship. The curve stays flat until around 90% sparsity before dropping off. Fine-tuning consistently helps, especially at higher sparsity where there is more accuracy to recover from.

4.2 Quantization Results

Table 3 tells a clean story. Dynamic quantization does little help, it shaves off a small amount of latency, 42.6ms to 35.5ms, and almost no model size. This makes sense: dynamic quantization only touches the Linear layers, which are tiny in our architecture thanks to global average pooling. The convulution layers, where most of the computation happens, are left untouched.

Static quantization had a larger impact. It compresses the model from 18.4 MB down to 4.65 MB (nearly 4×) and cuts the inference time from 42.6ms

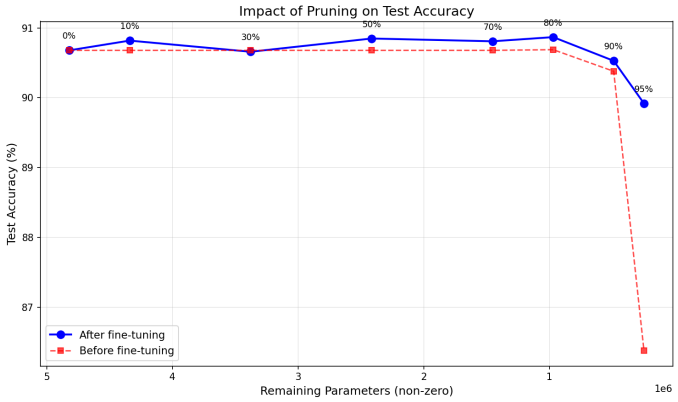

Figure 1: Test accuracy as a function of remaining (non-zero) parameters. The model retains over 90% accuracy even with only ∼488K of its original ∼4.82M parameters.

to 5.7ms around 7.5× faster. As well with small accuracy loss. In fact, the quantized model scored 90.70%, marginally above the FP32 baseline.

Table 3: Quantization comparison. Latency averaged over 200 runs on CPU, batch size 1.

Method	**Bits**	**Test Acc.**	**Size (MB)**	**Latency (ms)**	**Speedup**
FP32 Baseline	32	90.69%	18.43	42.61	1.0×
Dynamic INT8	8	90.67%	18.05	35.53	1.2×
Static INT8	8	90.70%	4.65	5.72	7.5×

4.3 Ablation Study

We ran three sets of experiments to learn what affects compression quality.

4.3.1 Global vs. Layer-wise Pruning

Does it matter whether we let the pruner decide where to remove weights, global or force every layer to lose the same fraction, layer-wise?

At low sparsity both strategies perform similarly at 30% and 50%. But the gap widens when sparsity is high. By 90% sparsity, global pruning maintains 90.56% accuracy while layer-wise collapses to 70.25%. At 95%, layer-wise is at a real low (36.65%).

The first conv block has only about 39K parameters but it's responsible for learning basic edge and texture detectors features the rest of the network builds on. When layer-wise pruning removes 90% of those, the Global pruning is

Table 4: Global vs. layer-wise pruning, both after 5 epochs of fine-tuning.

Sparsity	Global	Layer-wise	Gap
30%	90.67%	90.45%	+0.22%
50%	90.62%	90.13%	+0.49%
70%	90.75%	88.43%	+2.32%
80%	90.60%	86.02%	+4.58%
90%	90.56%	70.25%	+20.31%
95%	89.91%	36.65%	+53.26%

smarter: it recognizes that Block 4's 3.5M parameters have far more redundancy, so it concentrates the pruning there and largely leaves the early layers intact.

4.3.2 Fine-tuning Recovery

How quickly can fine tuning repair the damage from pruning, and does it depend on how much was removed? We tracked test accuracy epoch by epoch for 30 fine-tuning epochs at three sparsity levels: 90%, 95%, and 99%.

The results (Figure 2(b)) show three different behaviors:

- **90% sparsity:** The model starts at 90.38% and just hovers around 90.5% for all 30 epochs. There's essentially nothing to recover—the pruning didn't cause any real damage.
- **95% sparsity:** A bigger initial drop to 86.38%, but the model bounces back to 89.84% after just one epoch. After that it levels off near 90%, and the remaining 29 epochs add almost nothing.
- **99% sparsity:** The model is destroyed—accuracy starts at 10% (random chance on 10 classes). But with only ∼48K surviving weights, it gradually relearns: 70.6% after epoch 1, 81.4% by epoch 10, and 83.2% by epoch 30. The curve is still climbing at the end, suggesting more epochs could push it higher.

The best results seem to be between 95% and 99%. Below 95%, enough network structure survives for fast recovery. Above it 99%, the model has to essentially learn from scratch using the weights that remain.

4.3.3 Quantization Strategy

The dynamic vs static comparison (Table 3, Figure 2(c)) shows that picking the right quantization method matters a lot. Dynamic quantization's performance is affected from the design of the architecture of the model. Since the model uses global average pooling, the fully connected layers are small, and quantizing only those layers doesn't move the needle. Static quantization works on the entire model, including the conv layers where most of the computation lives, which is why it achieves dramatically better compression and speed.

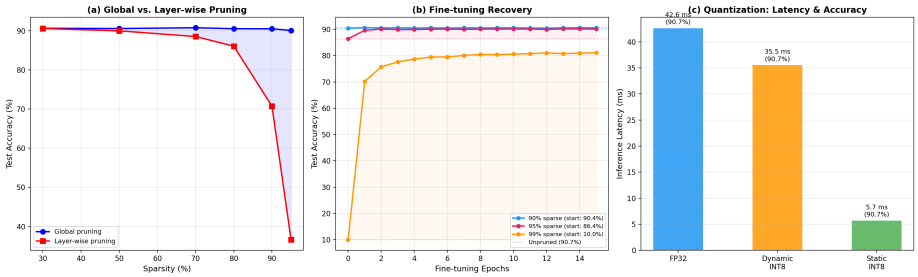

Figure 2: Ablation results. **(a)** Global pruning holds accuracy at high sparsity; layer-wise pruning collapses because it indiscriminately removes weights from sensitive early layers. **(b)** Fine-tuning recovery at 90% (flat—no damage), 95% (fast recovery in 1 epoch), and 99% (slow, steady climb that hasn't converged at 30 epochs). **(c)** Static quantization yields 7.5× speedup; dynamic quantization achieves only 1.2×.

5 THOUGHTS

The model is bigger than it needs to be.

is how much redundancy the network contains. You can throw away 90% of the weights and the model barely notices. Even at 99% sparsity keeping just 48K out of 4.8M weights—fine-tuning can recover to 83% accuracy. This level of overparameterization is consistent with the lottery ticket hypothesis [5], which suggests that large networks succeed partly because they contain many possible sparse subnetworks, any of which could have worked.

Where you prune matters more than how much.

The 53-point gap between global and layer-wise pruning at 95% sparsity is hard to overstate. It shows that treating all layers equally is a mistake—the early layers extract foundational features with relatively few parameters, so they're disproportionately sensitive to pruning. Global pruning sidesteps this by letting the algorithm figure out which layers can afford to lose more.

Pruning compresses in theory; quantization compresses in practice.

There's an important practical gap between the two methods. Our pruned models have fewer non-zero parameters and lower effective FLOPs, but they don't actually run faster because the tensors are still the same shape—the GPU processes every entry, zeros included. Using unstructured sparsity requires either structured pruning (physically removing filters) or specialized sparse inference libraries. Quantization, on the other hand, delivers immediate real-world benefits: the INT8 model is genuinely 4× smaller on disk and 7.5× faster to run on a standard CPU.

Combining both methods.

Since pruning and quantization compress in different ways—one removes weights, the other shrinks their representation—they could in principle be stacked. A pipeline that first prunes then quantizes the surviving weights might yield even

higher compression ratios than either method alone. We leave this direction for future work.

6 Conclusion

We compressed a VGG-style CNN on CIFAR-10 using global magnitude pruning and post-training static quantization. The main takeaways: 90% of the model's weights can be pruned with almost no accuracy cost, as long as pruning is done globally rather than layer by layer. Static INT8 quantization cuts model size by 4× and inference time by 7.5× on CPU, with no accuracy fall.

Fine-tuning after pruning shows three patterns at moderate sparsity there's nothing to fix, at high sparsity the model snaps back in one epoch, and at extreme sparsity it has to slowly relearn.

This suggests that the original network is far larger than it needs to be and that choosing the right compression strategy matters at least as much as choosing how aggressively to compress.

References

[1] `https://arxiv.org/abs/1409.1556` K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations (ICLR)*, 2015.

[2] S. Han, J. Pool, J. Tran, and W. Dally. Learning both weights and connections for efficient neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015. `https://arxiv.org/abs/1506.02626`

[3] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. `https://arxiv.org/abs/1712.05877`

[4] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf. Pruning filters for efficient convnets. In *International Conference on Learning Representations (ICLR)*, 2017. `https://arxiv.org/abs/1608.08710`

[5] J. Frankle and M. Carlin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations (ICLR)*, 2019. `https://arxiv.org/abs/1803.03635`

[6] M. Nagel, R. Amjad, M. Van Baalen, C. Louizos, and T. Blankevoort. Up or down? Adaptive rounding for post-training quantization. In *International Conference on Machine Learning (ICML)*, 2020. `https://arxiv.org/abs/2004.10568`

[7] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. https://arxiv.org/abs/1503.02531